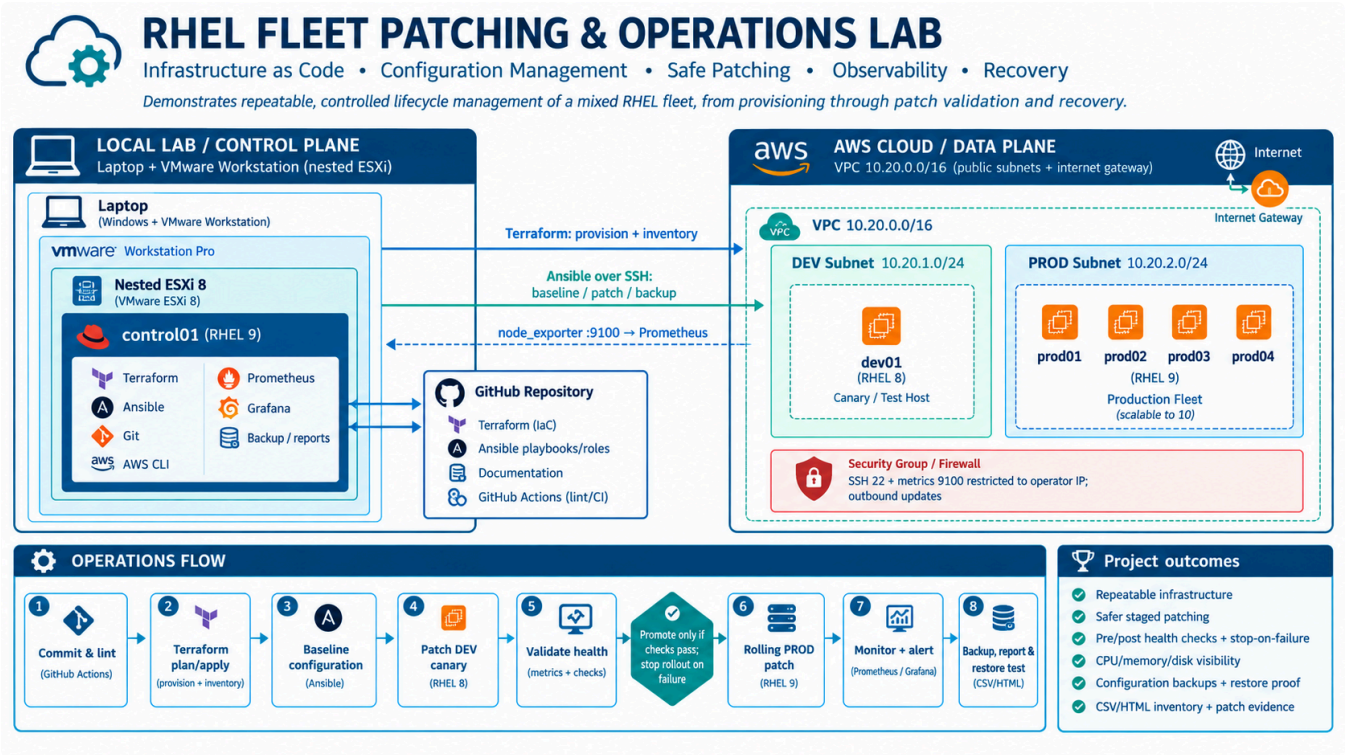
RHEL fleet patching home lab: step-by-step guide

Oct 7, 2026 · @no name for now

You will build a control node on VMware ESXi (nested in VMware Workstation on your laptop), use Terraform to create a fleet of RHEL servers on AWS EC2, then use Ansible to configure, patch, monitor, back up and report on that fleet. Budget two to three weekends. Every command is meant to be typed as shown; replace anything in `<angle brackets>`.

Status, 9 October 2026: the project was built end to end and the working code is in the GitHub repository `MamusIbebe/rhel-fleet-patching`. Steps 1 to 4b and the Step 5 snapshot task in this guide match what was run. The code shown in Steps 6, 7 and 8 was replaced during the build by simpler versions that suit the Ansible release shipped with RHEL 9; for those steps, and for the full patching role, use the files in the repository: `playbooks/`, `roles/` and `docs/`.

What the project does



`control01` is the only machine you work on. It runs on ESXi on your laptop, builds the AWS fleet with Terraform, and manages it with Ansible. The strip along the bottom shows the order of work: dev is patched and checked before prod, then monitoring, backups and reports follow.

What you need

This guide is sized for the laptop it was built on: an HP 15 with an Intel Core i3-8130U, 8 GB of RAM and about 70 GB of free disk. That is the minimum. It works because the fleet runs on AWS, but the laptop will be slow, so close every other program while the lab is running. With 16 GB of RAM, give `esxi01` 8 GB and `control01` 4 GB instead. You also need an internet connection; Wi-Fi is fine.

- **VMware Workstation Pro** on your Windows laptop (free for personal use).
- **The free VMware ESXi 8 ISO** from the Broadcom support portal (source). ESXi runs as a VM inside Workstation, called nested ESXi. It is the same product that runs on physical servers.
- **The RHEL 9 ISO** (x86_64 DVD) from developers.redhat.com for the control node, with your free Red Hat Developer login to register it.
- **An AWS account.** New accounts on the Free Plan receive credits that EC2 usage draws down (source). The AWS servers need no ISO and no Red Hat registration.
- **A GitHub account** and Git.

Machine	Where it runs	OS	Role	Size
esxi01	Workstation on the laptop	ESXi 8	Hypervisor	6 GB RAM, 2 cores, 80 GB disk
control01	ESXi	RHEL 9	Ansible, Terraform, Prometheus, Grafana	3 GB RAM, 2 CPUs, 30 GB disk
dev01	AWS EC2	RHEL 8	dev group	t3.micro
prod01 to prod10	AWS EC2	RHEL 9	prod group	t3.micro each

The number of prod servers is one Terraform variable, `prod_count`. This guide starts with 4 and you can raise it to 10. `dev01` runs RHEL 8 on purpose: it proves the same roles work across major versions, and it gives you a place to test patches before prod.

Laptop network. The lab uses Workstation's NAT network, VMnet8. On this laptop its subnet is `192.168.139.0`, shown under **Edit > Virtual Network Editor**. The gateway and DNS are always `.2` on that subnet, so `192.168.139.2`. ESXi is `192.168.139.5` and `control01` is `192.168.139.10`. On another computer, substitute its VMnet8 subnet wherever you see `192.168.139`. The AWS servers get public IPs, which Terraform writes into the Ansible inventory for you.

Cost. AWS bills by the hour while servers exist, and RHEL instances cost more than plain Linux. Set a budget alert in Step 2 and shut the fleet down at the end of every session as shown in Step 3.

Step 1: Install ESXi and build control01

Create the ESXi VM in Workstation, then build `control01` on it. Allow about an hour for ESXi and 20 minutes for the VM.

Part A: Install ESXi

1. Enable virtualization in the laptop's BIOS. Shut the laptop down, power it on, and tap **F10** until the BIOS setup opens. Under **Configuration** or **Advanced**, set **Virtualization Technology (VTx)** to Enabled, then save and exit with **F10**.
2. Turn off the Windows features that block Workstation. Press the Windows key, open **Turn Windows features on or off**, untick **Hyper-V**, **Virtual Machine Platform** and **Windows Hypervisor Platform** if they are ticked, click OK, and restart.
3. Download the free ESXi ISO. Log in at support.broadcom.com with a free account, open **My Downloads**, and click the word **HERE** in the blue bar that reads "Free Software Downloads available HERE". Search for **VMware vSphere Hypervisor**, tick the terms and conditions box, and download the ISO.
4. In Workstation choose **File > New Virtual Machine > Typical** and select the ESXi ISO. Workstation should detect it as VMware ESXi. Name the VM `esxi01` and give it an 80 GB disk stored as a single file.
5. Before finishing, click **Customize Hardware** and set:
   - **Memory:** 6 GB. If the installer reports too little memory, use 8 GB for the install and lower it afterwards.

- **Processors:** 2 cores, with **Virtualize Intel VT-x/EPT or AMD-V/RVI** ticked. Without this tick ESXi cannot run VMs of its own.
- **Network Adapter:** NAT. Host-only will not work, because the control node needs the internet to reach AWS.

6. Close other programs, power the VM on, and go through every installer screen: **Enter** to continue, **F11** to accept the licence, **Enter** to select the disk, **Enter** for the keyboard, a root password typed twice, then **F11** to install. Wait for the **Installation Complete** screen; do not restart the VM before it appears.
7. Remove the installer disc before rebooting. Press **Ctrl+Alt** to release the mouse, choose **VM > Removable Devices > CD/DVD > Disconnect**, and untick **Connect at power on** under **VM > Settings > CD/DVD**. Then press **Enter** in the VM to reboot.
8. Wait, without pressing any key, until the yellow and grey ESXi screen appears. Pressing F2 too early opens the VM's firmware menu instead. Then press **F2**, log in as `root`, and open **Configure Management Network > IPv4 Configuration**. Select **Set static IPv4 address** with the space bar and enter IP `192.168.139.5`, netmask `255.255.255.0`, gateway `192.168.139.2`. Under **DNS Configuration** set `192.168.139.2`. Press **Esc**, then **Y** to apply.
9. In a browser on the laptop, go to `https://192.168.139.5`, accept the certificate warning, and log in as `root`. This web page is the ESXi Host Client; you manage ESXi from here.
10. Upload the RHEL 9 ISO: go to **Storage > datastore1 > Datastore browser > Upload**. Afterwards delete the copy in your Downloads folder to free about 10 GB.
11. If you have disk space to spare, take a Workstation snapshot of `esxi01` named `esxi-clean` (**VM > Snapshot > Take Snapshot**), so you can return to a fresh ESXi at any time.

If ESXi has no datastore. On a boot disk this small, ESXi 8 keeps the whole disk for its own system files and creates no datastore, so step 10 fails or the datastore browser shows an error. Fix it with a second disk:

1. Shut `esxi01` down. In Workstation choose **VM > Settings > Add > Hard Disk**, pick **SCSI**, create a new 50 GB disk stored as a single file, and leave **Allocate all disk space now** unticked.
2. Power `esxi01` on. In the Host Client go to **Storage > Datastores > New datastore > Create new VMFS datastore**, name it `datastore1`, select the new disk, and choose **Use full disk** with VMFS 6.

3. Enable SSH under **Host > Actions > Services > Enable Secure Shell (SSH)**, then copy the ISO from Windows PowerShell, which is more reliable than the browser for a 14 GB file:

```
scp -O "C:\ISO\rhel-9.8-x86_64-dvd.iso"
root@192.168.139.5:/vmfs/volumes/datastore1/
```

If the datastore ended up with another name, rename it with `ssh root@192.168.139.5 "vim-cmd hostsvc/datastore/rename '<old name>' datastore1"`.

Part B: Create control01

1. In the Host Client choose **Virtual Machines > Create / Register VM > Create a new virtual machine**. Name it `control01`, set guest OS family to **Linux**, and version to **Red Hat Enterprise Linux 9 (64-bit)**.
2. On the settings page give it 2 CPUs, 3 GB of RAM and a 30 GB disk set to **Thin provisioned**. For **CD/DVD Drive 1** choose **Datastore ISO file**, pick the RHEL 9 ISO, and tick **Connect at power on**. Leave the network adapter on **VM Network**.
3. Power the VM on and open its console. In the installer pick **Minimal Install**, set a root password, and create an admin user named `ansible` with **Make this user administrator** ticked.
4. Under **Network & Host Name**, turn the adapter on, set the hostname to `control01`, then open **Configure > IPv4**, choose **Manual**, and enter IP `192.168.139.10`, netmask `255.255.255.0`, and `192.168.139.2` as both gateway and DNS.
5. Finish the install and reboot. Log in as `ansible` and confirm networking works:

```
ip -4 addr show        # shows the static IP you set
ping -c 2 redhat.com
```

Then register the VM with your Red Hat login. Until you do, `dnf` cannot download anything and every later step fails with "This system is not registered":

```
sudo subscription-manager register --username <your-red-hat-login>
sudo dnf repolist      # should list BaseOS and AppStream
```

6. Shut `control01` down once with `sudo poweroff`. In the Host Client select the VM, choose **Actions > Snapshots > Take snapshot**, name it `clean-install`, then power it back on. Take another snapshot here before any risky change to the control node.

Step 2: Set up the control node

The control node needs four tools (Ansible, Terraform, Git and the AWS command line), an SSH key, and credentials for your AWS account. Commands run on `control01` as the `ansible` user; the AWS console clicks happen in a browser on your laptop.

1. Install the tools:

```
sudo dnf install -y ansible-core git python3-pip rsync dnf-plugins-core
sudo dnf config-manager --add-repo
https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
sudo dnf install -y terraform
pip3 install --user ansible-lint yamllint awscli boto3 botocore
```

2. Let the `ansible` user run sudo without a password on `control01`, because later steps configure the control node itself:

```
echo 'ansible ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/ansible
```

3. Create the SSH key that will log in to every AWS server:

```
ssh-keygen -t ed25519 -N '' -f ~/.ssh/id_ed25519
```

4. Create your AWS account at aws.amazon.com and choose the **Free Plan**. Then open **Billing and Cost Management > Budgets > Create budget** and create a small monthly budget (for example $10) with an email alert.
5. Create credentials for Terraform. In the AWS console open **IAM > Users > Create user**, name it `terraform-lab`, and attach the `AmazonEC2FullAccess` policy directly. Open the new user, choose **Security credentials > Create access key > Command Line Interface**, and copy the access key ID and the secret.
6. Give the credentials to `control01` and check that they work:

```
aws configure                 # paste key ID and secret; region us-east-1;
output json
aws sts get-caller-identity   # should print your account number
```

The access key is stored in `~/.aws`, outside the repo. Never paste it into a file you commit. In a real company you would use short-lived role credentials instead of a long-lived key; say so in your README.

Step 3: Create the repo and build the fleet with Terraform

Terraform describes the AWS servers as code, creates them, and writes the Ansible inventory for you. Change one number and the fleet grows from 4 prod servers to 10.

1. Create an empty repo named `rhel-fleet-patching` on GitHub, then on `control01`:

```
git config --global user.name "<Your Name>"
git config --global user.email "<you@example.com>"
git clone https://github.com/<your-username>/rhel-fleet-patching.git
cd rhel-fleet-patching
mkdir -p terraform inventories/{dev,prod} playbooks roles docs reports
.github/workflows
```

2. Create `ansible.cfg` in the repo root. AWS RHEL servers log in as `ec2-user`:

```
[defaults]
inventory = inventories/dev/hosts.yml
roles_path = roles
remote_user = ec2-user
host_key_checking = False
callback_result_format = yaml

[privilege_escalation]
become = True
```

3. Create `requirements.yml` and install the Ansible collections the later steps use:

```
collections:
  - name: ansible.posix
  - name: community.general
  - name: prometheus.prometheus
  - name: amazon.aws
```

```
ansible-galaxy collection install -r requirements.yml
```

4. Create `terraform/main.tf`:

The file builds a dedicated network for the lab instead of using the AWS default one: a VPC (`10.20.0.0/16`), a dev subnet and a prod subnet, an internet gateway, a route table, and a security group that admits only your IP address.

```
terraform {
  required_providers {
    aws   = { source = "hashicorp/aws", version = ">= 5.0" }
    http  = { source = "hashicorp/http" }
    local = { source = "hashicorp/local" }
  }
}

variable "region" {
  default = "us-east-1"
}

variable "instance_type" {
  default = "t3.micro"
}

variable "prod_count" {
  default = 4
  validation {
    condition     = var.prod_count >= 1 && var.prod_count <= 10
    error_message = "prod_count must be between 1 and 10."
  }
}

variable "vpc_cidr" {
  default = "10.20.0.0/16"
}

variable "dev_subnet_cidr" {
  default = "10.20.1.0/24"
}

variable "prod_subnet_cidr" {
  default = "10.20.2.0/24"
}

provider "aws" {
  region = var.region
  default_tags {
    tags = { project = "rhel-fleet-patching" }
  }
}
```

```
# Your current public IP: only this address may reach the servers
data "http" "my_ip" {
  url = "https://checkip.amazonaws.com"
}

data "aws_availability_zones" "available" {
  state = "available"
}

locals {
  my_cidr    = "${chomp(data.http.my_ip.response_body)}/32"
  prod_names = [for i in range(var.prod_count) : format("prod%02d", i + 1)]
  az         = data.aws_availability_zones.available.names[0]
}

# ---------------------------------------------------------------- network
# A dedicated VPC for the lab, with one subnet for dev and one for prod
resource "aws_vpc" "lab" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags                 = { Name = "rhel-fleet-vpc" }
}

resource "aws_internet_gateway" "lab" {
  vpc_id = aws_vpc.lab.id
  tags   = { Name = "rhel-fleet-igw" }
}

resource "aws_subnet" "dev" {
  vpc_id                  = aws_vpc.lab.id
  cidr_block              = var.dev_subnet_cidr
  availability_zone       = local.az
  map_public_ip_on_launch = true
  tags                    = { Name = "rhel-fleet-dev", env = "dev" }
}

resource "aws_subnet" "prod" {
  vpc_id                  = aws_vpc.lab.id
  cidr_block              = var.prod_subnet_cidr
  availability_zone       = local.az
  map_public_ip_on_launch = true
  tags                    = { Name = "rhel-fleet-prod", env = "prod" }
```

```
}

# Route table: send internet-bound traffic through the internet gateway
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.lab.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.lab.id
  }
  tags = { Name = "rhel-fleet-public" }
}

resource "aws_route_table_association" "dev" {
  subnet_id      = aws_subnet.dev.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "prod" {
  subnet_id      = aws_subnet.prod.id
  route_table_id = aws_route_table.public.id
}

# Firewall: SSH and metrics from your IP only, everything outbound
resource "aws_security_group" "lab" {
  name        = "rhel-fleet-sg"
  description = "SSH and metrics from my IP only"
  vpc_id      = aws_vpc.lab.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [local.my_cidr]
  }

  ingress {
    description = "node_exporter"
    from_port   = 9100
    to_port     = 9100
    protocol    = "tcp"
    cidr_blocks = [local.my_cidr]
  }
```

```
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "rhel-fleet-sg" }
}

# ------------------------------------------------------------------ servers
# Latest official Red Hat images (309956199498 is Red Hat's AWS account)
data "aws_ami" "rhel8" {
  most_recent = true
  owners      = ["309956199498"]
  filter {
    name   = "name"
    values = ["RHEL-8.*_HVM-*-x86_64-*-Hourly2-GP3"]
  }
}

data "aws_ami" "rhel9" {
  most_recent = true
  owners      = ["309956199498"]
  filter {
    name   = "name"
    values = ["RHEL-9.*_HVM-*-x86_64-*-Hourly2-GP3"]
  }
}

resource "aws_key_pair" "lab" {
  key_name   = "rhel-fleet-lab"
  public_key = file(pathexpand("~/.ssh/id_ed25519.pub"))
}

resource "aws_instance" "dev" {
  ami                    = data.aws_ami.rhel8.id
  instance_type          = var.instance_type
  key_name               = aws_key_pair.lab.key_name
  subnet_id              = aws_subnet.dev.id
  vpc_security_group_ids = [aws_security_group.lab.id]
  tags                   = { Name = "dev01", env = "dev" }
```

```
  depends_on             = [aws_route_table_association.dev]
}

resource "aws_instance" "prod" {
  count                  = var.prod_count
  ami                    = data.aws_ami.rhel9.id
  instance_type          = var.instance_type
  key_name               = aws_key_pair.lab.key_name
  subnet_id              = aws_subnet.prod.id
  vpc_security_group_ids = [aws_security_group.lab.id]
  tags                   = { Name = local.prod_names[count.index], env =
"prod" }
  depends_on             = [aws_route_table_association.prod]
}

# ------------------------------------------------------------- inventories
# Ansible inventories, written from what was actually built
resource "local_file" "dev_inventory" {
  filename        = "${path.module}/../inventories/dev/hosts.yml"
  file_permission = "0644"
  content = yamlencode({
    all = {
      vars = { env = "dev", aws_region = var.region }
      children = { linux = { hosts = {
        dev01 = { ansible_host = aws_instance.dev.public_ip, instance_id =
aws_instance.dev.id }
      } } }
    }
  })
}

resource "local_file" "prod_inventory" {
  filename        = "${path.module}/../inventories/prod/hosts.yml"
  file_permission = "0644"
  content = yamlencode({
    all = {
      vars = { env = "prod", aws_region = var.region }
      children = { linux = { hosts = {
        for i, inst in aws_instance.prod :
        local.prod_names[i] => { ansible_host = inst.public_ip, instance_id =
inst.id }
      } } }
    }
```

```
  })
}

resource "local_file" "monitoring_targets" {
  filename        = "${path.module}/../inventories/monitoring_targets.yml"
  file_permission = "0644"
  content = yamlencode({
    monitoring_targets = [
      for ip in concat([aws_instance.dev.public_ip],
aws_instance.prod[*].public_ip) : "${ip}:9100"
    ]
  })
}

output "instance_ids" {
  value = join(" ", concat([aws_instance.dev.id], aws_instance.prod[*].id))
}

output "vpc_id" {
  value = aws_vpc.lab.id
}
```

5. Build the fleet. Read the plan before you type `yes`:

```
cd terraform
terraform init
terraform plan
terraform apply
cd ..
cat inventories/prod/hosts.yml
```

6. Wait a minute for the servers to boot, then test both environments. Each host should answer `pong`:

```
ansible linux -m ping
ansible linux -m ping -i inventories/prod/hosts.yml
```

7. Grow the fleet when you are ready. Terraform adds only the missing servers and rewrites the inventory:

```
terraform -chdir=terraform apply -var prod_count=10
```

8. End every session by stopping the servers, which keeps their disks, or destroying them, which removes everything:

```
# Pause: keeps disks and patch history, small storage charge
aws ec2 stop-instances --instance-ids $(terraform -chdir=terraform output -
raw instance_ids)

# Resume: start them, then refresh the inventory because public IPs change
aws ec2 start-instances --instance-ids $(terraform -chdir=terraform output -
raw instance_ids)
terraform -chdir=terraform apply

# Finished for good: delete every server
terraform -chdir=terraform destroy
```

Because `ansible.cfg` defaults to the dev inventory, every command runs against dev unless you add `-i inventories/prod/hosts.yml`. That is your safety catch. Commit with `git add . && git commit -m "Add Terraform fleet and ansible.cfg" && git push`; Step 9 keeps Terraform's state file and the generated inventories out of the repo.

Step 4: The baseline role

A role is a reusable bundle of tasks. This one brings every server to the same known-good configuration, which is what makes patching predictable.

1. Create the folders:

```
mkdir -p roles/baseline/{tasks,handlers,defaults}
```

2. Create `roles/baseline/tasks/main.yml`:

The baseline follows the CIS Benchmark for RHEL, Level 1 Server, and each task name carries the NIST SP 800-53 control it supports. First create `roles/baseline/defaults/main.yml` with the settings:

```
---
baseline_banner: "Authorized users only. All activity may be monitored and
reported.\n"

baseline_sshd_settings:
  PermitRootLogin: "no"
  PasswordAuthentication: "no"
```

```
  PermitEmptyPasswords: "no"
  HostbasedAuthentication: "no"
  IgnoreRhosts: "yes"
  X11Forwarding: "no"
  MaxAuthTries: "4"
  LoginGraceTime: "60"
  ClientAliveInterval: "300"
  ClientAliveCountMax: "3"
  LogLevel: INFO
  Banner: /etc/issue.net

baseline_sysctl:
  net.ipv4.conf.all.accept_redirects: "0"
  net.ipv4.conf.default.accept_redirects: "0"
  net.ipv4.conf.all.send_redirects: "0"
  net.ipv4.conf.default.send_redirects: "0"
  net.ipv4.conf.all.accept_source_route: "0"
  net.ipv4.conf.default.accept_source_route: "0"
  net.ipv4.conf.all.log_martians: "1"
  net.ipv4.conf.default.log_martians: "1"
  net.ipv4.conf.all.rp_filter: "1"
  net.ipv4.icmp_echo_ignore_broadcasts: "1"
  net.ipv4.tcp_syncookies: "1"
  kernel.randomize_va_space: "2"
```

Then the tasks:

```
---
# Each task name ends with the NIST SP 800-53 control it supports.

- name: Install baseline packages (CM-6)
  ansible.builtin.dnf:
    name: [chrony, firewalld, audit, rsync, tar, yum-utils]
    state: present

- name: Enable time sync, firewall and audit logging (AU-8, SC-7, AU-12)
  ansible.builtin.service:
    name: "{{ item }}"
    state: started
    enabled: true
  loop: [chronyd, firewalld, auditd]

- name: Keep SELinux enforcing (AC-3, AC-6)
```

```
  ansible.posix.selinux:
    policy: targeted
    state: enforcing

- name: Set the login warning banner (AC-8)
  ansible.builtin.copy:
    dest: "{{ item }}"
    content: "{{ baseline_banner }}"
    mode: "0644"
  loop: [/etc/issue, /etc/issue.net]

- name: Set the message of the day
  ansible.builtin.copy:
    dest: /etc/motd
    content: "{{ inventory_hostname }} ({{ env }}) - managed by Ansible\n"
    mode: "0644"

# Version-specific: EL9 reads drop-in files first, EL8 has only the main file
- name: Harden SSH with a drop-in file, EL9 (AC-17, IA-2, AC-12)
  ansible.builtin.copy:
    dest: /etc/ssh/sshd_config.d/00-baseline.conf
    content: |
      # Managed by Ansible - baseline role
      {% for key, value in baseline_sshd_settings.items() %}
      {{ key }} {{ value }}
      {% endfor %}
    mode: "0600"
    validate: /usr/sbin/sshd -t -f %s
  when: ansible_facts.distribution_major_version | int >= 9
  notify: Restart sshd

- name: Harden SSH in sshd_config, EL8 (AC-17, IA-2, AC-12)
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: "^#?\\s*{{ item.key }}\\s"
    line: "{{ item.key }} {{ item.value }}"
    validate: /usr/sbin/sshd -t -f %s
  loop: "{{ baseline_sshd_settings | dict2items }}"
  when: ansible_facts.distribution_major_version == "8"
  notify: Restart sshd

- name: Restrict sshd_config permissions (AC-6)
  ansible.builtin.file:
```

```
    path: /etc/ssh/sshd_config
    owner: root
    group: root
    mode: "0600"

- name: Apply kernel network and memory hardening (SC-5, SC-7, SI-16)
  ansible.posix.sysctl:
    name: "{{ item.key }}"
    value: "{{ item.value }}"
    sysctl_file: /etc/sysctl.d/60-baseline.conf
    sysctl_set: true
    reload: true
  loop: "{{ baseline_sysctl | dict2items }}"

- name: Disable unused filesystem modules (CM-7)
  ansible.builtin.copy:
    dest: /etc/modprobe.d/baseline-disable-fs.conf
    content: |
      install cramfs /bin/false
      blacklist cramfs
      install udf /bin/false
      blacklist udf
    mode: "0644"

# Version-specific: EL9 already rejects SHA-1, EL8 needs it switched off
- name: Read crypto policy, EL8 (SC-13)
  ansible.builtin.command: update-crypto-policies --show
  register: baseline_crypto
  changed_when: false
  when: ansible_facts.distribution_major_version == "8"

- name: Disable SHA-1, EL8 (SC-13)
  ansible.builtin.command: update-crypto-policies --set DEFAULT:NO-SHA1
  changed_when: true
  when:
    - ansible_facts.distribution_major_version == "8"
    - baseline_crypto.stdout != "DEFAULT:NO-SHA1"
```

3. Create `roles/baseline/handlers/main.yml`:

```
---
- name: Restart sshd
  ansible.builtin.service:
    name: sshd
    state: restarted
```

4. Create `playbooks/site.yml`:

```
---
- name: Apply baseline configuration
  hosts: linux
  roles:
    - swap
    - baseline
```

The `swap` role comes first because a 1 GB `t3.micro` runs out of memory during package installs ("Cannot allocate memory"). Create it with `mkdir -p roles/swap/tasks` and put this in `roles/swap/tasks/main.yml`:

```
---
- name: Check for active swap
  ansible.builtin.command: swapon --noheadings --show
  register: swap_active
  changed_when: false

- name: Create the swap file
  ansible.builtin.shell: |
    set -e
    dd if=/dev/zero of=/swapfile bs=1M count=2048
    chmod 600 /swapfile
    mkswap /swapfile
  args:
    creates: /swapfile

- name: Activate the swap file
  ansible.builtin.command: swapon /swapfile
  changed_when: true
  when: "'/swapfile' not in swap_active.stdout"

- name: Keep swap across reboots
```

```
  ansible.builtin.lineinfile:
    path: /etc/fstab
    regexp: "^/swapfile\\s"
    line: "/swapfile none swap sw 0 0"
```

List `swap` first in every playbook that installs packages, including `patch.yml` in Step 5.

5. Run it on dev, then prod. Use `--check --diff` first to preview changes without making them:

```
ansible-playbook playbooks/site.yml --check --diff
ansible-playbook playbooks/site.yml
ansible-playbook playbooks/site.yml -i inventories/prod/hosts.yml
```

6. Run it a second time. Every task should report `ok` and none `changed`. That property is called idempotence, and it is worth a screenshot for your README.

Commit: `git add . && git commit -m "Add baseline role" && git push`.

Step 4b: Measure the baseline against CIS

A baseline is only a claim until it is measured. This step scans every server with OpenSCAP against the CIS Level 1 Server profile that ships with RHEL, and records a score you can show before and after hardening.

CIS is the right starting point because it is a set of actual server settings. NIST SP 800-53, the NIST Cybersecurity Framework, FedRAMP and ISO 27001 are control frameworks that those settings support; `docs/compliance-mapping.md` in the repo maps each project feature to its NIST control.

1. Create the folders with `mkdir -p roles/compliance/{tasks,defaults} reports/compliance`.
2. Create `roles/compliance/defaults/main.yml`:

```
---
# Other profiles shipped with RHEL: cis (Level 2), stig, cui (NIST SP 800-
171), pci-dss, hipaa
compliance_profile: cis_server_l1
```

3. Create `roles/compliance/tasks/main.yml`:

```
---
- name: Install OpenSCAP and the SCAP Security Guide
  ansible.builtin.dnf:
    name: [openscap-scanner, scap-security-guide]
    state: present

- name: Run the compliance scan (several minutes per server)
  ansible.builtin.command: >-
    oscap xccdf eval
    --profile xccdf_org.ssgproject.content_profile_{{ compliance_profile }}
    --results /var/tmp/compliance-results.xml
    --report /var/tmp/compliance-report.html
    /usr/share/xml/scap/ssg/content/ssg-rhel{{
ansible_facts.distribution_major_version }}-ds.xml
  register: compliance_scan
  changed_when: false
  # oscap returns 0 when every rule passes and 2 when some fail; 1 is a real
error
  failed_when: compliance_scan.rc not in [0, 2]

- name: Count passed and failed rules
  ansible.builtin.set_fact:
    compliance_pass: "{{ compliance_scan.stdout |
regex_findall('Result\\s+pass') | length }}"
    compliance_fail: "{{ compliance_scan.stdout |
regex_findall('Result\\s+fail') | length }}"

- name: Work out the score
  ansible.builtin.set_fact:
    compliance_score: >-
      {{ (100 * (compliance_pass | int) / ([(compliance_pass | int) +
(compliance_fail | int), 1] | max)) | round(1) }}

- name: Show the score
  ansible.builtin.debug:
    msg: >-
      {{ inventory_hostname }}: {{ compliance_pass }} passed, {{
compliance_fail }} failed
      ({{ compliance_score }}% of checked rules) against {{
compliance_profile }}

- name: Copy the HTML report to the control node
  ansible.builtin.fetch:
```

```
    src: /var/tmp/compliance-report.html
    dest: "{{ playbook_dir }}/../reports/compliance/{{ inventory_hostname }}-
{{ compliance_profile }}-{{ ansible_facts.date_time.date }}.html"
    flat: true

- name: Record the score on the control node
  ansible.builtin.lineinfile:
    path: "{{ playbook_dir }}/../reports/compliance-log.csv"
    line: "{{ ansible_facts.date_time.iso8601 }},{{ inventory_hostname }},{{
env }},{{ compliance_profile }},{{ compliance_pass }},{{ compliance_fail }},
{{ compliance_score }}"
    create: true
    mode: "0644"
  delegate_to: localhost
  become: false
  throttle: 1
```

4. Create `playbooks/compliance_scan.yml`:

```
---
- name: Measure servers against the compliance profile
  hosts: linux
  roles:
    - swap
    - compliance
```

5. Scan before applying the baseline, apply it, then scan again. The difference between the two scores is your evidence:

```
ansible-playbook playbooks/compliance_scan.yml
ansible-playbook playbooks/site.yml
ansible-playbook playbooks/compliance_scan.yml
cat reports/compliance-log.csv
```

6. Repeat for prod with `-i inventories/prod/hosts.yml`. Open an HTML report from `reports/compliance/` in your browser to see every rule, its result and the fix for each failure.

Measured on `dev01` (RHEL 8.10, `t3.micro`) on 9 October 2026:

Stage	Passed	Failed	High-severity failures	OpenSCAP score
Stock AWS image	183	115	3	78.66%
After the baseline role	207	93	2	82.07%
After round two (password policy, AIDE)	220	80	1	83.26%

The four prod servers (RHEL 9) went from 154 passed and 106 failed (76.42%) on the stock image to 192 passed and 70 failed (80.02%) after the same baseline. A second run of `site.yml` on prod reported `changed=0` on every server, which shows the role is idempotent.

The remaining failures are mostly password and account policy, file integrity monitoring (AIDE) and disk partition layout.

The score will not reach 100%. This lab applies a subset of CIS Level 1; rules such as disk partition layout, password ageing and full audit rules are left for a production build. Do not run the scanner's automatic fix-everything mode: some rules restrict sudo and can lock Ansible out. Add fixes to the baseline role one at a time and rescan.

Step 5: The patching playbook

This is the centerpiece: check health, snapshot, patch one server at a time, reboot only if needed, check health again, and stop the rollout if anything fails.

1. Create the folders:

```
mkdir -p roles/patching/{tasks,defaults}
```

2. Create `roles/patching/defaults/main.yml` (settings you can override per run):

```
---
patching_security_only: true
patching_min_free_mb: 1024
patching_critical_services: [sshd, chronyd, firewalld, auditd]
patching_ec2_snapshot: true
```

3. Create `roles/patching/tasks/main.yml`:

```
---
- name: Pre-checks
  ansible.builtin.import_tasks: precheck.yml

- name: Snapshot
  ansible.builtin.import_tasks: snapshot.yml

- name: Patch
  ansible.builtin.import_tasks: patch.yml

- name: Post-checks
  ansible.builtin.import_tasks: postcheck.yml
```

4. Create `roles/patching/tasks/precheck.yml`:

```
---
- name: Check free space on /
  ansible.builtin.assert:
    that: >-
      (ansible_facts.mounts | selectattr('mount', 'equalto', '/') |
first).size_available
      > patching_min_free_mb * 1024 * 1024
    fail_msg: "Not enough free space on / to patch safely"

- name: Count failed systemd units before patching
  ansible.builtin.command: systemctl --failed --no-legend
  register: pre_failed
  changed_when: false

- name: Remember the running kernel
  ansible.builtin.set_fact:
    pre_kernel: "{{ ansible_facts.kernel }}"
```

5. Create `roles/patching/tasks/snapshot.yml`. It runs the AWS command-line tool on the control node to snapshot each server's disk before touching it, using the instance ID that Terraform wrote into the inventory. The AWS CLI is used here because the newest `amazon.aws` Ansible collection does not support the Ansible version that ships with RHEL 9:

```
---
- name: Snapshot the server's disk before patching
  ansible.builtin.command:
    argv:
      - aws
      - ec2
      - create-snapshots
      - --region
      - "{{ aws_region }}"
      - --instance-specification
      - "InstanceId={{ instance_id }}"
      - --description
      - "pre-patch {{ inventory_hostname }} {{
ansible_facts.date_time.iso8601 }}"
      - --tag-specifications
      - "ResourceType=snapshot,Tags=[{Key=Name,Value=pre-patch-{{
inventory_hostname }}},{Key=project,Value=rhel-fleet-patching}]"
      - --query
      - "Snapshots[0].SnapshotId"
      - --output
      - text
  register: patching_snapshot
  changed_when: true
  delegate_to: localhost
  become: false
  when: patching_ec2_snapshot | bool

- name: Show the snapshot ID
  ansible.builtin.debug:
    msg: "{{ inventory_hostname }} snapshot: {{ patching_snapshot.stdout }}"
  when: patching_ec2_snapshot | bool
```

6. Create `roles/patching/tasks/patch.yml`:

```
---
- name: Apply updates
  ansible.builtin.dnf:
    name: "*"
    state: latest
    security: "{{ patching_security_only }}"
    update_cache: true
  register: patch_result

- name: Check whether a reboot is required
  ansible.builtin.command: needs-restarting -r
  register: reboot_check
  changed_when: false
  failed_when: reboot_check.rc not in [0, 1]

- name: Reboot if required
  ansible.builtin.reboot:
    reboot_timeout: 600
  when: reboot_check.rc == 1
```

7. Create `roles/patching/tasks/postcheck.yml`:

```
---
- name: Refresh facts after patching
  ansible.builtin.setup:

- name: Gather service state
  ansible.builtin.service_facts:

- name: Critical services must be running
  ansible.builtin.assert:
    that: >-
      (ansible_facts.services[item ~ '.service'] | default({})).state
      | default('missing') == 'running'
    fail_msg: "{{ item }} is not running after patching"
  loop: "{{ patching_critical_services }}"

- name: Count failed systemd units after patching
  ansible.builtin.command: systemctl --failed --no-legend
  register: post_failed
  changed_when: false
```

```
- name: No new failed units
  ansible.builtin.assert:
    that: post_failed.stdout_lines | length <= pre_failed.stdout_lines |
length
    fail_msg: "New failed units: {{ post_failed.stdout }}"

- name: Record the result on the control node
  ansible.builtin.lineinfile:
    path: "{{ playbook_dir }}/../reports/patch-log.csv"
    line: >-
      {{ ansible_facts.date_time.iso8601 }},{{ inventory_hostname }},{{ env
}},{{ pre_kernel }},{{ ansible_facts.kernel }},{{ patch_result.changed }},{{
reboot_check.rc == 1 }}
    create: true
    mode: "0644"
  delegate_to: localhost
  become: false
```

8. Create `playbooks/patch.yml`. The `serial` list patches one server first as a canary, then the rest in batches of half the fleet, and `max_fail_percentage: 0` halts the rollout on the first failure:

```
---
- name: Rolling patching
  hosts: linux
  serial:
    - 1
    - "50%"
  max_fail_percentage: 0
  roles:
    - swap
    - patching
```

9. Patch dev, confirm it is healthy, then patch prod:

```
ansible-playbook playbooks/patch.yml
ansible-playbook playbooks/patch.yml -i inventories/prod/hosts.yml
cat reports/patch-log.csv
```

10. Prove the safety stop works. This run pretends a service that does not exist is critical, so the canary `prod01` fails its post-check and no other prod server is touched:

```
ansible-playbook playbooks/patch.yml -i inventories/prod/hosts.yml \
  -e '{"patching_critical_services": ["sshd", "nosuchservice"]}'
```

Screenshot both the clean run and the halted run. If a fresh VM has no security updates pending, add `-e patching_security_only=false` to apply all updates and see a reboot.

About the snapshots: each run leaves one snapshot per server under **EC2 > Snapshots** in the AWS console. To roll a server back you create a volume from its snapshot and swap it in for the damaged one; write that procedure into your failure runbook. Snapshots cost a little to store and `terraform destroy` does not remove them, so delete old ones.

Step 6: The inventory report

This playbook collects facts from every server and writes one CSV and one HTML file, a small stand-in for keeping a configuration management database accurate.

1. Create `playbooks/inventory_report.yml`:

```
---
- name: Collect facts from every server
  hosts: linux
  tasks:
    - name: Find the date of the last package change
      ansible.builtin.shell: set -o pipefail && rpm -qa --last | head -1 |
awk '{print $3, $4, $5}'
      args:
        executable: /bin/bash
      register: last_patch
      changed_when: false

- name: Build the report on the control node
  hosts: localhost
  connection: local
  become: false
  gather_facts: false
  tasks:
    - name: Write CSV and HTML reports
      ansible.builtin.template:
        src: "inventory.{{ item }}.j2"
```

```
        dest: "{{ playbook_dir }}/../reports/inventory-{{
hostvars[groups['linux'][0]].env }}.{{ item }}"
        mode: "0644"
      loop: [csv, html]
```

2. Create `playbooks/templates/inventory.csv.j2` (run `mkdir -p playbooks/templates` first):

```
hostname,env,ip,os,kernel,vcpus,ram_mb,uptime_days,last_patched
{% for h in groups['linux'] %}
{% set f = hostvars[h].ansible_facts %}
{{ h }},{{ hostvars[h].env }},{{ f.default_ipv4.address }},{{ f.distribution
}} {{ f.distribution_version }},{{ f.kernel }},{{ f.processor_vcpus }},{{
f.memtotal_mb }},{{ (f.uptime_seconds / 86400) | round(1) }},{{
hostvars[h].last_patch.stdout }}
{% endfor %}
```

3. Create `playbooks/templates/inventory.html.j2`:

```
<html><body>
<h1>Server inventory</h1>
<table border="1" cellpadding="6">
<tr><th>Host</th><th>Env</th><th>IP</th><th>OS</th><th>Kernel</th>
<th>vCPU</th><th>RAM MB</th><th>Uptime days</th><th>Last patched</th></tr>
{% for h in groups['linux'] %}
{% set f = hostvars[h].ansible_facts %}
<tr><td>{{ h }}</td><td>{{ hostvars[h].env }}</td><td>{{
f.default_ipv4.address }}</td><td>{{ f.distribution }} {{
f.distribution_version }}</td><td>{{ f.kernel }}</td><td>{{ f.processor_vcpus
}}</td><td>{{ f.memtotal_mb }}</td><td>{{ (f.uptime_seconds / 86400) |
round(1) }}</td><td>{{ hostvars[h].last_patch.stdout }}</td></tr>
{% endfor %}
</table>
</body></html>
```

4. Run it for both environments and read the result:

```
ansible-playbook playbooks/inventory_report.yml
ansible-playbook playbooks/inventory_report.yml -i inventories/prod/hosts.yml
cat reports/inventory-prod.csv
```

Commit the generated reports to the repo as sample output. Run the report before and after a patch run and the kernel and last-patched columns show the change.

Step 7: Monitoring

Each target runs node_exporter, a small agent that publishes CPU, memory and disk metrics on port 9100. Prometheus on `control01` collects them and Grafana draws the dashboards.

1. This step makes changes on `control01` itself, so give its `ansible` user passwordless sudo too:

```
echo 'ansible ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/ansible
```

2. Create `playbooks/monitoring.yml`. The list of servers to watch comes from the file Terraform generated, so it always matches the fleet:

```
---
- name: Install node_exporter on every server
  hosts: linux
  roles:
    - prometheus.prometheus.node_exporter
  tasks:
    - name: Open the metrics port
      ansible.posix.firewalld:
        port: 9100/tcp
        permanent: true
        immediate: true
        state: enabled

- name: Install Prometheus and Grafana on the control node
  hosts: localhost
  connection: local
  vars_files:
    - ../inventories/monitoring_targets.yml
  vars:
    prometheus_scrape_configs:
      - job_name: node
        static_configs:
          - targets: "{{ monitoring_targets }}"
  roles:
    - prometheus.prometheus.prometheus
  tasks:
    - name: Add the Grafana repository
      ansible.builtin.yum_repository:
        name: grafana
```

```
        description: Grafana OSS
        baseurl: https://rpm.grafana.com
        gpgcheck: true
        gpgkey: https://rpm.grafana.com/gpg.key

    - name: Install Grafana
      ansible.builtin.dnf:
        name: grafana
        state: present

    - name: Start Grafana
      ansible.builtin.service:
        name: grafana-server
        state: started
        enabled: true

    - name: Open Prometheus and Grafana ports
      ansible.posix.firewalld:
        port: "{{ item }}"
        permanent: true
        immediate: true
        state: enabled
      loop: [9090/tcp, 3000/tcp]
```

3. Run it against dev, then prod. The second play repeats harmlessly:

```
ansible-playbook playbooks/monitoring.yml
ansible-playbook playbooks/monitoring.yml -i inventories/prod/hosts.yml
```

4. On your laptop, open `http://192.168.139.10:9090/targets`. Every server in the fleet should show **UP**.
5. Open `http://192.168.139.10:3000` and log in as `admin` / `admin` (it asks you to set a new password).
6. Go to **Connections > Data sources > Add data source > Prometheus**, set the URL to `http://localhost:9090`, and click **Save & test**.
7. Go to **Dashboards > New > Import**, enter ID `1860` (Node Exporter Full), pick your Prometheus data source, and click **Import**.
8. Test an alert. Stop the agent on dev with `ansible dev01 -a "systemctl stop node_exporter"`, wait a few minutes, and check `http://192.168.139.10:9090/alerts`. The Prometheus role ships with default alert

rules, including one for a host that stops responding. Start the agent again with `ansible dev01 -a "systemctl start node_exporter"`.

9. Rerun this playbook whenever the fleet changes size or the servers come back with new IPs.

Screenshot the targets page, the dashboard and the firing alert. The roles download their binaries through the control node, so it needs internet access during this step.

Step 8: Backups and a tested restore

Each night the control node pulls a compressed copy of `/etc` from every server. A backup only counts once you have restored from it, so this step ends with a restore test.

1. Create the backup store on `control01`:

```
sudo mkdir -p /srv/backups && sudo chown ansible: /srv/backups
mkdir -p roles/backup/tasks
```

2. Create `roles/backup/tasks/main.yml`:

```
---
- name: Archive /etc
  community.general.archive:
    path: /etc
    dest: "/var/tmp/etc-{{ ansible_facts.date_time.date }}.tar.gz"
    mode: "0600"

- name: Copy the archive to the control node
  ansible.builtin.fetch:
    src: "/var/tmp/etc-{{ ansible_facts.date_time.date }}.tar.gz"
    dest: "/srv/backups/{{ inventory_hostname }}/"
    flat: true

- name: Remove the copy left on the server
  ansible.builtin.file:
    path: "/var/tmp/etc-{{ ansible_facts.date_time.date }}.tar.gz"
    state: absent
```

3. Create `playbooks/backup.yml`:

```
---
- name: Back up server configuration
  hosts: linux
  roles:
    - backup
```

4. Run it and confirm the files arrived:

```
ansible-playbook playbooks/backup.yml
ansible-playbook playbooks/backup.yml -i inventories/prod/hosts.yml
ls -lh /srv/backups/*/
```

5. Schedule it nightly at 02:00. Run `crontab -e` and add:

```
0 2 * * * cd ~/rhel-fleet-patching && ansible-playbook playbooks/backup.yml
>> ~/backup.log 2>&1
15 2 * * * cd ~/rhel-fleet-patching && ansible-playbook playbooks/backup.yml
-i inventories/prod/hosts.yml >> ~/backup.log 2>&1
```

6. Test a restore. Break a file on `dev01`, then bring it back from the archive:

```
ansible dev01 -m shell -a "echo broken > /etc/motd"
cd /srv/backups/dev01
tar -tzf etc-<date>.tar.gz | grep motd      # shows the path inside the
archive
tar -xzf etc-<date>.tar.gz -C /tmp etc/motd
cd ~/rhel-fleet-patching
ansible dev01 -m copy -a "src=/tmp/etc/motd dest=/etc/motd"
ansible dev01 -a "cat /etc/motd"
```

If the path printed by `tar -tzf` differs from `etc/motd`, use the path it shows. Write these restore steps into `docs/runbook-restore.md`.

Step 9: Docs, CI and publishing

The documentation is what turns a working lab into evidence a hiring manager can read in two minutes.

1. Create `.gitignore` so secrets and scratch files stay out of the repo:

```
*.retry
vault.yml
.vault_pass
.terraform/
*.tfstate
*.tfstate.*
*.tfvars
inventories/dev/hosts.yml
inventories/prod/hosts.yml
inventories/monitoring_targets.yml
```

2. Create `.yamllint`:

```
extends: default
rules:
  line-length: disable
  truthy:
    check-keys: false
```

3. Create `.ansible-lint`:

```
skip_list:
  - package-latest               # patching intentionally installs the latest
packages
  - var-naming[no-role-prefix]
```

4. Run both linters locally and fix what they report. Anything you choose to skip, add to the skip list with a comment saying why:

```
yamllint .
ansible-lint
```

5. Create `.github/workflows/lint.yml` so GitHub runs the same checks on every push:

```
name: lint
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: hashicorp/setup-terraform@v3
      - run: pip install ansible-core ansible-lint yamllint
      - run: ansible-galaxy collection install -r requirements.yml
      - run: yamllint .
      - run: ansible-lint
      - run: terraform -chdir=terraform fmt -check
```

6. Write three short documents in `docs/`:

File	What goes in it
`change-record-example.md`	A sample change request for the prod patch run: scope, schedule, risk, rollback plan (revert to the `pre-patch` snapshot), validation steps, and who was notified
`runbook-patch-failure.md`	What to do when a host fails its post-check: read the failed task, check `systemctl --failed` and `journalctl -b`, decide fix-forward or snapshot rollback, then resume with `--limit <host>`
`runbook-restore.md`	The restore steps you tested in Step 8

7. Write `README.md` with these parts, in this order:
   - One paragraph on what the project does and why
   - The architecture diagram from this guide (save it as `docs/images/architecture.png`)
   - A table mapping each feature to a line in the job description
   - How to run it: the commands from Steps 4 to 8

- Screenshots: idempotent baseline run, clean patch run, halted patch run, Grafana dashboard, firing alert, inventory report
- Lab versus production: what you would change at scale (Red Hat Satellite for content, AWX or Ansible Automation Platform for scheduling, a vault for credentials, ServiceNow for change records, vCenter for snapshots, physical ESXi hosts in place of the nested one)

8. Push everything and check the **Actions** tab on GitHub for a green tick:

```
git add . && git commit -m "Add docs, lint config and CI" && git push
```

Commit after each step of this guide instead of all at once. A history of small, named commits shows how you work.

Troubleshooting

Most beginner failures come from networking or SSH, so check those first.

Symptom	Likely cause	Fix
Workstation says virtualized Intel VT-x/EPT or AMD-V is not supported	Windows Hyper-V or Memory integrity is holding the CPU's virtualization features	In Windows turn off **Memory integrity** (Windows Security > Device security > Core isolation) and the Hyper-V and Virtual Machine Platform features, then reboot
control01 has no internet	Wrong gateway or DNS	Both should end in `.2` on your VMnet8 subnet; check with `nmcli device show`
Terraform: `No valid credential sources found`	AWS credentials not set up	Rerun `aws configure` from Step 2
Terraform: `UnauthorizedOperation`	The IAM user lacks EC2 permissions	Attach `AmazonEC2FullAccess` to `terraform-lab`
Terraform: `Your query returned no results` for an image	The image name filter matches nothing in your region	Loosen the filter in `main.tf`, for example `RHEL-9.*x86_64*`

Symptom	Likely cause	Fix
Terraform complains about a missing default VPC	The account or region has no default network	In the VPC console choose **Actions > Create default VPC**
`UNREACHABLE` on ping	Servers still booting, public IPs changed after a restart, or your home IP changed	Wait a minute, then run `terraform -chdir=terraform apply` to refresh the inventory and the firewall rule
`Permission denied (publickey)`	Wrong user or key	`remote_user` must be `ec2-user`, and the key must be `~/.ssh/id_ed25519`
Nothing updates during patching	A fresh AWS image is already up to date	Add `-e patching_security_only=false`, or stop the servers between sessions instead of destroying them so updates build up
Snapshot task fails	AWS Python library or collection missing	`pip3 install --user boto3 botocore`, then `ansible-galaxy collection install -r requirements.yml`
Module fails on dev01 with a Python error	Newer Ansible dropped support for the old Python on RHEL 8	Use the `ansible-core` package from RHEL 9 as in Step 2, not a newer pip install
Prometheus target shows DOWN	Port 9100 closed, node_exporter stopped, or an old IP list	Run `terraform -chdir=terraform apply`, then rerun the monitoring playbook
Cannot open Grafana from your laptop	Port 3000 closed on control01	`sudo firewall-cmd --list-ports`; rerun the monitoring playbook
AWS charges higher than expected	Servers left running, or old snapshots	Stop or destroy the fleet as in Step 3, delete snapshots under **EC2 > Snapshots**, and check **Billing**

Symptom	Likely cause	Fix
esxi01 shows "Attempting to start up from" and every device is unsuccessful	The ESXi install did not finish before the reboot	Power off, reconnect the ESXi ISO under VM > Settings > CD/DVD, and run the installer through to Installation Complete
A blue Boot Maintenance Manager menu appears instead of ESXi	F2 was pressed while the VM was still starting	Restart the VM and press nothing until the yellow and grey ESXi screen appears
ESXi shows an address outside the VMnet8 subnet	The VM's network adapter is on Host-only instead of NAT	Set VM > Settings > Network Adapter to NAT, then set the static address again

Run any playbook with `-v` for more detail, or `--limit <host>` to target one server. When a cloud server goes badly wrong, run `terraform destroy`, `terraform apply`, then `site.yml`. Rebuilding from code is the point of the project.